ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

HPC Lab for CSE

FS 2026
Dr. R. Käppeli,
Dr. F. Gatti, J. Bachmann,
H. Schrader, A. Pietak,
S. Utzinger, V. Vogt,
B. Zemann.

Project 7

Due date: **Friday** 29 May 2026, 18:00 (Europe/Zurich). Please carefully review the information regarding the deadline on the course webpage. Follow the instructions in Section A.

This project introduces the elementary architectural and performance characteristics of Graphics Processing Units (GPUs) and general-purpose computing on GPUs (GPGPU) using NVIDIA's CUDA (Compute Unified Device Architecture) Application Programming Interface (API).

Our choice to use NVIDIA's CUDA is primarily due to its availability on the Euler cluster. Other manufacturers provide analogous GPU or, more generally, accelerator technology APIs. For example, AMD features the ROCm software stack with the Heterogeneous Interface for Portability (HIP) API. Likewise, Intel's oneAPI is a unified programming model for CPUs, GPUs, and other accelerators, using the DPC++ and SYCL APIs.

Compiler-directives-based APIs provide an alternative high-level, pragma-driven approach to parallel programming, allowing developers to annotate their code with pragmas to specify parallel regions. For example, OpenACC is designed for accelerators like GPUs, enabling portable performance across different hardware by using directives to offload computations. Similarly, OpenMP extends support beyond multi-core CPUs to include accelerators, providing pragmas for shared-memory parallelism and offloading. Both APIs abstract away much of the complexity of low-level parallel programming, making them more accessible and portable for scientific computing.

To summarize, this project serves only as an introduction to basic GPU and accelerator programming. We encourage you to continue exploring these technologies in more advanced courses or through self-study. Although the following project description should be self-contained, we highly recommend the CUDA training series [7] for the basics and more advanced subjects beyond this course.

For the whole project, the simulations must be run on the Euler GPU nodes with NVIDIA RTX 2080 Ti GPUs (for which we have privileged access during the lab hours), and we will use the default software stack using the GNU Compiler Collection and NVIDIA's `nvcc` compiler:

```
user@eu-login-36:~$ module load stack gcc cuda
user@eu-login-36:~$ gcc --version
gcc (Spack GCC) 12.2.0
Copyright (C) 2022 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
user@eu-login-36:~$ nvcc --version
nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed_Aug_20_01:58:59_PM_PDT_2025
Cuda compilation tools, release 13.0, V13.0.88
Build cuda_13.0.r13.0/compiler.36424714_0
user@eu-login-36:~$ srun --gpus=rtx_2080:1 ./program # Run command with a job
                                                     # requiring a specific GPU
srun: job 65782908 queued and waiting for resources
srun: job 65782908 has been allocated resources
...
```

However, feel free to try and develop on other available systems (e.g., your workstation or laptop) and compilers, but please make sure to document them in your report if you include results.

1 GPU architecture and performance characteristics [15 Points]

GPUs can provide higher instruction throughput and memory bandwidth than CPUs with a comparable energy consumption. They are specialized for highly parallel computations and their design is such that

more transistors are devoted to data processing rather than data caching and flow control. By devoting more transistors to data processing (in the form of "simpler" compute cores), GPUs are specialized for highly parallel computations such that they can hide memory access latencies with computation.

As we know, it is crucial to understand the performance limits of computing devices. In Project 1, we explored these limits in the intuitive roofline performance model for a classical multi-core CPU system. Likewise, the goal of this task is to familiarize you with the GPU architecture and their performance characteristics.

1.1 Peak performance [5 Points]

In this task, you are asked to compute the peak performance of an NVIDIA RTX 2080 Ti, for both single-precision and double-precision arithmetic. Please document your computations in detail and cite all sources you use in your report. Below, we provide some context and a step-by-step guide to help you complete and report the task.

In scientific computing, the primary focus is often on floating-point (FP) data, typically using *double precision* (FP64). However, many GPUs exhibit significantly higher performance for smaller FP formats, such as *single precision* (FP32). Therefore, it is meaningful to report peak performance numbers for both single and double precision. The rate at which a GPU can produce results for multiplication and addition operations is quantified in terms of *floating-point operations per second* (Flops/s, or simply FLOPS). The maximum rate at which a system is capable of executing Flops/s is the so-called (theoretical[1]) *peak (FP) performance*. The theoretical peak floating-point performance of a GPU can be estimated as

$$P_{\text{GPU}} = n_{\text{SMs}} \times \frac{n_{\text{cores per SM}}}{TP_{FP}} \times n_{\text{FMA}} \times f, \tag{1}$$

where

- n_{SMs} is the number of Streaming Multiprocessors (SMs),
- $n_{\text{cores per SM}}$ is the number of CUDA cores per SM,
- TP_{FP} is a throughput scaling factor reflecting that not all CUDA cores can operate on every FP format with full throughput,
- n_{FMA} is the fused multiply-add factor: $n_{\text{FMA}} = 2$ if the CUDA core supports the FMA instruction (enabling it to perform a multiplication and an addition in a single instruction), or $n_{\text{FMA}} = 1$ if FMA is not supported (requiring separate instructions for multiplication and addition),
- f is the GPU clock frequency in Hz.

Figure 1 shows a simplified view of the architecture of an NVIDIA GTX 1080 Ti GPU. An NVIDIA GPU consists of n_{SMs} *Streaming Multiprocessors* (SMs). Each SM is designed to execute thousands of lightweight threads in parallel. Within an SM, there are $n_{\text{cores per SM}}$ *CUDA cores*, which are simple arithmetic units capable of performing floating-point (addition, multiplication, FMA, etc.), integer, and logical operations. CUDA cores primarily handle single-precision (FP32) arithmetic, while double-precision (FP64) operations are typically executed by a smaller set of dedicated FP64-capable units, resulting in reduced double-precision throughput compared to single-precision. Threads are organized into groups of 32 called *warps*. A warp is the fundamental unit of execution in the CUDA model: all threads in a warp execute the same instruction simultaneously, following the *Single Instruction, Multiple Threads* (SIMT) paradigm. SIMT can be seen as a combination of Single Instruction, Multiple Data (SIMD) execution with multithreading concepts.

As an example, let's compute the peak performance of an NVIDIA GTX 1080 Ti[2]. In order to compute P_{GPU} for a GPU, the factors in Eq. (1) need to be determined. However, these details typically require a deep dive into technical documents or utility commands/scripts provided by the GPU manufacturer, or from technology review sites and academic research. NVIDIA provides the utility `deviceQuery` [4] from which the needed information can be extracted[3]. This is shown in Listing 1, from which we immediately extract that the NVIDIA GTX 1080 Ti has

- $n_{\text{SMs}} = 28$ SMs,

[1] There is a distinction between theoretical and measured peak performance, but we will ignore it for the time being (see [8] for more information).

[2] The NVIDIA GTX 1080 Ti was launched in 2017 and the installed specimen on Euler cluster have been decommissioned in 2023-2024.

[3] Note that you need to obtain it from the repository and build it.

- $n_{\text{cores per SM}} = 128$ CUDA cores per SM,
- $f = 1.58$ GHz (max) GPU clock frequency.

Next, we have to determine the FP capabilities of the CUDA cores and their throughput factor for different FP formats. For NVIDIA GPUs, this is encoded in the CUDA GPU Compute Capability (CC). We extract from Listing 1 that the NVIDIA GTX 1080 Ti has CC 6.1, which we can confirm with NVIDIA's *Legacy CUDA GPU CC table* [1]. This places the GPU into the *NVIDIA Pascal GPU architecture*. The specifications for each CC are documented in [3, Table 28 in Sec. 18., Compute Capabilities[4]] However, [3, Table 7 in Sec. 6.4.1., Arithmetic Instructions[5]] provides a convenient way to extract the necessary information:

- For single precision (FP32) FMAs $TP_{FP32} = 1$,
- For double precision (FP64) FMAs $TP_{FP64} = 32$.

Note that the above cites the archived v12.9 version. For newer GPU generations, we refer to the current version [5, Sec. 5.1, Compute Capabilities[6]] and [2, Table 5 in Sec. 12.1.1., Throughput of Native Arithmetic Instructions[7]].

In summary, we can compute the (theoretical) peak performance of an NVIDIA GTX 1080 Ti for single and double precision FP formats as

$$P_{\text{FP32}} = 28 \times \frac{128}{1} \times 2 \text{ FP32s} \times 1.58 \text{ GHz} = 11325.44 \text{ GFlops/s},$$
$$P_{\text{FP64}} = 28 \times \frac{128}{32} \times 2 \text{ FP64s} \times 1.58 \text{ GHz} = 353.92 \text{ GFlops/s}.$$

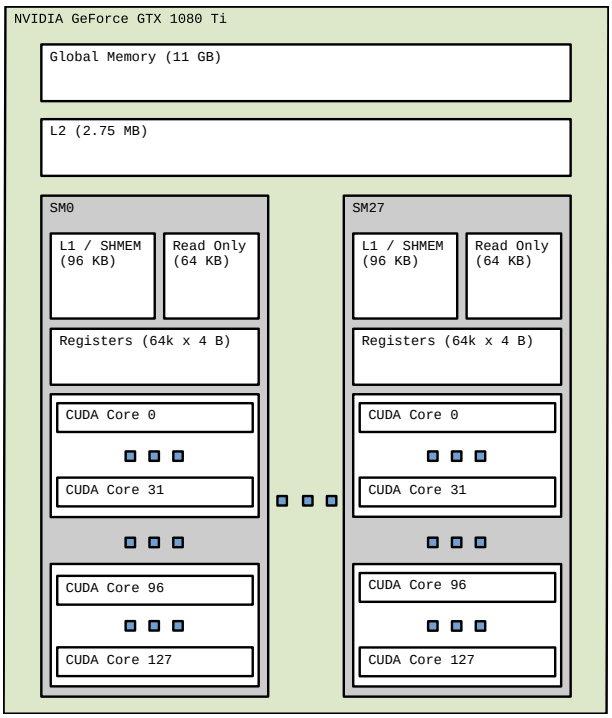


Figure 1: Simplified GPU architecture for an NVIDIA GTX 1080 Ti.

1.2 Memory hierarchies and bandwidth [5 Points]

In this task, you are asked to collect basic size and bandwidth information of an NVIDIA RTX 2080 Ti GPU. Please document your computations in detail and cite all sources you use in your report. Below, we provide some background and a step-by-step guide to help you complete and report the task.

[4] https://docs.nvidia.com/cuda/archive/12.9.0/cuda-c-programming-guide/index.html#compute-capabilities
[5] https://docs.nvidia.com/cuda/archive/12.9.0/cuda-c-programming-guide/index.html#arithmetic-instructions
[6] https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capabilities.html
[7] https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#arithmetic-instructions-throughput-native-arithmetic-instructions

```
user@eu-login-12:~$ module load stack gcc cuda cmake # load a stack, a compiler,
                                                     # CUDA and cmake
user@eu-login-12:~$ git clone ...; cmake ... # obtain and build deviceQuery
user@eu-login-12:~$ srun --gpus=gtx_1080:1 ./deviceQuery # Run command with a
                                                         # job requiring a
                                                         # specific GPU
srun: job 65783416 queued and waiting for resources
srun: job 65783416 has been allocated resources
.../deviceQuery Starting...

 CUDA Device Query (Runtime API)

Detected 1 CUDA Capable device(s)

Device 0: "NVIDIA GeForce GTX 1080 Ti"
  ...
  CUDA Capability Major/Minor version number:    6.1
  Total amount of global memory:                 11264 MBytes (11810963456 bytes)
  (28) Multiprocessors, (128) CUDA Cores/MP:     3584 CUDA Cores
  GPU Max Clock rate:                            1582 MHz (1.58 GHz)
  Memory Clock rate:                             5505 Mhz
  Memory Bus Width:                              352-bit
  L2 Cache Size:                                 2883584 bytes
  ...
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total number of registers available per block: 65536
  ...
```

Listing 1: Example of running `deviceQuery` on the Euler cluster to extract an NVIDIA GPU's performance characteristics.

Efficient use of memory is also a critical aspect of achieving high performance on GPUs. Similar to CPUs, memory is organized in a hierarchy of fast but small memories and slow but large ones. The memory hierarchy of an NVIDIA GTX 1080 Ti GPU is shown in Fig. 1. It consists of the following:

- **Global Memory**: Large off-chip memory accessible by all threads. High latency, high bandwidth, used for general data storage.
- **L2 Cache**: Device-wide cache shared by all SMs. Reduces global memory access latency.
- **L1 Cache and Shared Memory**: On-chip memory at the SM level. L1 Cache: Automatically caches global memory accesses. Shared Memory: Programmer-managed, fast, used for data sharing within a block. The sizes can be configured, but the maximal size is set by the CC / architecture (see, e.g., the [3, Table 28 in Sec. 18., Compute Capabilities[8]]).
- **Read-Only Memory (Constant and Texture)**: Specialized cached memories for specific access patterns. Constant Memory: Fast for broadcast of small data. Texture Memory: Optimized for spatial locality.
- **Registers**: Fast, private storage for each thread. Extremely low latency, highest bandwidth, limited in size. `deviceQuery` outputs the number of registers per block, which is the number of threads running on an SM. Each register can hold one single precision number, i.e., 4 bytes (B).

Each level offers a trade-off between capacity, latency, and bandwidth. Efficient GPU programming aims to maximize usage of low-latency, high-bandwidth memories.

The (theoretical) peak memory bandwidth of a GPU quantifies the maximum rate at which data can be transferred between global memory and the compute units. It is determined by the properties of the memory subsystem and can be estimated as:

$$B_{\text{GPU}} = \frac{W_{\text{mem}}}{8} \times f_{\text{mem}} \times 2, \tag{2}$$

[8] https://docs.nvidia.com/cuda/archive/12.9.0/cuda-c-programming-guide/index.html#compute-capabilities

where

- W_{mem} is the memory bus width in bits. It specifies how many bits can be transferred in parallel per memory clock cycle.
- f_{mem} is the memory clock frequency in Hz.
- The factor of 2 accounts for double data rate (DDR) memory, which transfers data on both rising and falling edges of the clock.

The sizes of the memories in the hierarchy for the NVIDIA GTX 1080 Ti can be readily extracted from the `deviceQuery` output Listing 1:

- **Global Memory**: 11 GB.
- **L2 Cache**: 2.75 MB.
- **L1 and Shared Memory per SM**: 96 KB.
- **Read-Only Memory**: 64 KB.
- **Registers**: 64 K registers per block, each holding 4 bytes (total 256 KB).

The total size of L1 and Shared Memory per SM for the CC 6.1 Pascal architecture can be found in the [3, Sec. 18.5. Compute Capability 6.x[9]]. The bandwidth is obtained from the memory bus width $W_{\text{mem}} = 352$ bits and the memory clock frequency $f_{\text{mem}} = 7$ GHh:

$$B_{\text{max}} = \frac{352}{8} \times 5.505 \text{ GHz} \times 2 = 484.44 \text{ GB/s}.$$

1.3 Performance model: A simple roofline model [5 Points]

With your performance data collected in the previous tasks,

1. Create a simple roofline model for the NVIDIA RTX 2080 Ti with single and double precision.
2. Visualize your roofline models in one single plot.
3. At what operational intensity is a kernel or application memory/compute-bound?

Include your plotting scripts in your submission.
For example, for an NVIDIA GTX 1080 Ti the roofline model is shown in Fig. 2.

2 Introduction to CUDA [50 Points]

CUDA (Compute Unified Device Architecture) is NVIDIA's parallel computing platform and application programming interface (API) to leverage the massive parallel computing power of GPUs. It builds on a minimal set of extensions to the C/C++ language and a runtime library. There are essentially three key abstractions that are exposed to the programmer:

- **Hierarchy of Thread Groups**: A CUDA program consists of *kernels*, which are functions executed in parallel by many lightweight *threads* on the GPU. Threads are organized into *blocks*, which are further organized into a *grid*. This hierarchy maps directly to the hardware:
  - A *thread* executes on a CUDA core.
  - A *block* of threads is assigned to an SM.
  - A *grid* of blocks spans the entire GPU.
- **Shared Memory and Thread Cooperation**: Threads within a block can cooperate by exchanging data through *shared memory* on an SM, a low-latency, on-chip memory accessible by all threads in the block. By reusing data in shared memory, threads reduce costly accesses to global memory and improve overall performance. Shared memory is explicitly managed by the programmer and is a key tool for optimizing memory-bound workloads.

[9] https://docs.nvidia.com/cuda/archive/12.9.0/cuda-c-programming-guide/index.html#compute-capability-6-x

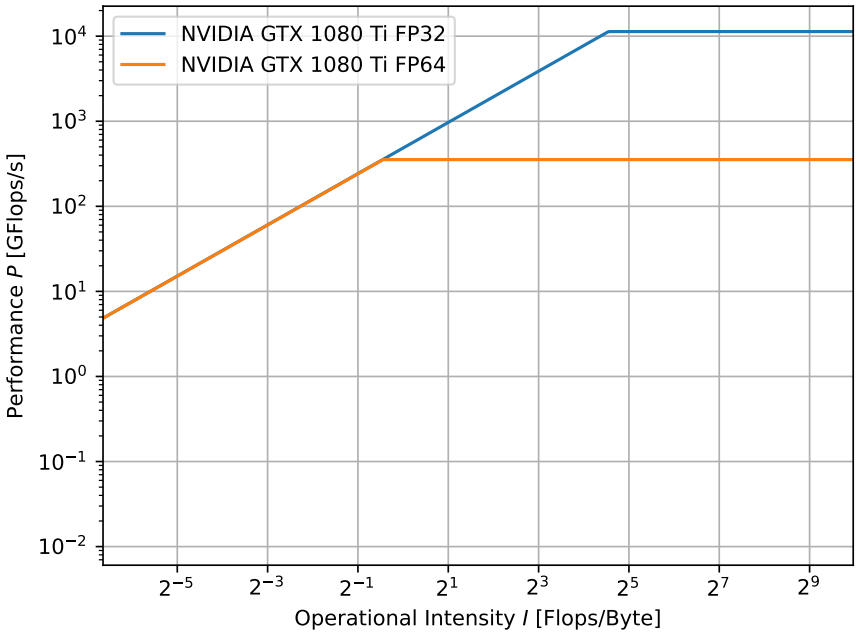


Figure 2: Simple (or naive) roofline model for an NVIDIA GTX 1080 Ti.

- **Barrier Synchronization**: Threads within a block can synchronize their execution using barrier primitives. This ensures that all threads reach a specific point in the program before any thread proceeds, which is essential when threads communicate via shared memory or need to coordinate access to shared data. Barrier synchronization is only available within a block, as threads from different blocks run on different SMs and cannot synchronize directly.

These abstractions enable *fine-grained parallelism*, where many lightweight threads execute similar operations on different data elements (*data parallelism*), and synchronize within a block (*thread parallelism*). This is embedded within *coarse-grained parallelism*, where independent blocks execute concurrently across multiple SMs, corresponding to *task or block-level parallelism*.

2.1 Hello world: Vector and matrix add

CUDA extends C/C++ by allowing the programmer to define so-called *kernel functions* (or simply *kernels*), which are executed in parallel by a number of threads on the GPU. A kernel is defined using the `__global__` declaration specifier, which marks the function as executable on the *device* (GPU) and callable from the *host* (CPU). A kernel is launched from the host by calling it as a function, augmented with an *execution configuration*, which specifies how many threads and blocks are used for execution. The execution configuration is provided using the syntax `<<<blocks, threads>>>`, where `blocks` specifies the number of blocks in the grid, and `threads` specifies the number of threads per block.

2.1.1 Vector add

As an example, we consider a CUDA program that adds two vectors of `float` elements. The example can be found in `skeleton_codes/vec_add`. It demonstrates how to write a CUDA kernel, how to launch it with a given number of threads, and how threads identify which part of the data they are responsible for. The `vec_add` kernel function is shown in Listing 2. The key elements are:

- The keyword `__global__` declares this function as a CUDA kernel executed on the device and callable from the host ([5, Sec. 5.4.1.1.[10]]).
- Each thread executes this function independently and is responsible for computing exactly one element of the result vector.

[10] https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#execution-space-specifiers

- The index `i` of the element processed by a thread is computed using CUDA's built-in variables ([5, Sec. 5.4.2.2.[11]])

  ```
  i = blockIdx.x * blockDim.x + threadIdx.x
  ```

  where

  - `blockIdx` is the index of the current block within the grid ,
  - `blockDim` is the number of threads per block ,
  - `threadIdx` is the index of the thread within its block .

- The conditional `if (i < N)` ensures that threads beyond the data range do not perform invalid accesses.

```
__global__ void vec_add(const float* a, const float* b, float* c, int N) {
  // Compute index with CUDA built-in variables blockIdx, blockDim and threadIdx
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  // Avoid out-of-bound
  if (i < N) {
    c[i] = a[i] + b[i];
  }
}
```

Listing 2: CUDA kernel for addition of two vectors. Each thread computes one element of the result vector by determining its global index from the grid and block dimensions.

The vector addition kernel is used in the `main` program shown in Listing 3. The core components are:

- The constant `N = 512` defines the size of the vectors to be processed.
- The host pointers `a`, `b`, `c` reside in CPU memory.
- The device pointers `d_a`, `d_b`, `d_c` reside in GPU memory. We add the prefix `d_`, but this convention is arbitrary.
- The host vectors are allocated using `malloc()` and initialized.
- The device vectors allocated for using `cudaMalloc()` [6, manpage].
- The host vectors `a` and `b` are copied to the device vectors with `cudaMemcpy()` [6, manpage] and the `cudaMemcpyHostToDevice` [6, manpage] flag.
- The kernel `vec_add` is launched on the device. The execution configuration is specified with `<<<blocks, threads>>>`, where:
  - `threads = 256` specifies the number of threads per block.
  - `blocks = (N + threads - 1) / threads` computes the number of blocks needed to cover all elements.
- The result vector `d_c` is copied back from device to host memory in `c` using `cudaMemcpy()` with the `cudaMemcpyDeviceToHost` flag.
- Device memory is freed with `cudaFree()` [6, manpage].
- Host memory is freed with `free()`.

Note that the CUDA `cuda*` API functions are declared in `cuda_runtime.h`.
CUDA code is compiled with NVIDIA's `nvcc` compiler and run as follows:

```
user@eu-login-12:~$ module load stack gcc cuda # load a stack, a compiler and CUDA
user@eu-login-12:~$ nvcc -arch=sm_75 vec_add.cu -o vec_add # compile for CC 7.5
user@eu-login-12:~$ srun --gpus=rtx_2080:1 ./vec_add # Run command with a job
                                                     # requiring a specific GPU
srun: job 31822488 queued and waiting for resources
srun: job 31822488 has been allocated resources
Results match! GPU computation is correct :-).
```

[11] https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#built-in-variables

```cpp
int main(int argc, char* argv[]) {
  // Size of vectors
  const int N = 512;
  // Host copies of vectors a, b, c (i.e., living on "CPU")
  float *a, *b, *c;
  // Device copies of vectors a, b, c (with arbitrary convention d_*) (i.e.,
  // living on "GPU")
  float *d_a, *d_b, *d_c;

  // Allocate vectors on host and initialize
  a = (float *)malloc(N * sizeof(float));
  b = (float *)malloc(N * sizeof(float));
  c = (float *)malloc(N * sizeof(float));

  // Initialize vectors on host
  for (int i = 0; i < N; ++i) {
    a[i] = i; b[i] = N - i; c[i] = 0.;
  }

  // Allocate vectors on device
  cudaMalloc((void **)&d_a, N * sizeof(float));
  cudaMalloc((void **)&d_b, N * sizeof(float));
  cudaMalloc((void **)&d_c, N * sizeof(float));

  // Copy host vectors to device vectors
  cudaMemcpy(d_a, a, N * sizeof(float), cudaMemcpyHostToDevice);
  cudaMemcpy(d_b, b, N * sizeof(float), cudaMemcpyHostToDevice);

  // Launch kernel on device
  int threads = 256;
  int blocks = (N + threads - 1) / threads;
  // "threads" threads will execute vec_add as a block on a SM
  // "blocks" blocks of threads execute as a grid on the device
  vec_add<<<blocks, threads>>>(d_a, d_b, d_c, N);

  // Copy device result c vector to host
  cudaMemcpy(c, d_c, N * sizeof(float), cudaMemcpyDeviceToHost);

  // Check results
  bool success = true;
  float feps = std::numeric_limits<float>::epsilon();
  for (int i = 0; i < N; ++i) {
    if (std::abs(a[i] + b[i] - c[i]) > feps) {
      std::cerr << "Mismatch at index " << i << ": GPU result " << c[i]
                << ", expected " << a[i] + b[i] << "\n";
      success = false;
    }
  }
  if (success) {
    std::cout << "Results match! GPU computation is correct :-).\n";
  } else {
    std::cerr << "Results do not match :-(!\n";
  }

  // Deallocate device vectors
  cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);

  // Deallocate host vectors
  free(a); free(b); free(c);

  return 0;
}
```

Listing 3: Host-side `main` function for vector addition on the GPU.

2.1.2 Matrix add

Building on the vector addition example, we now extend the concept to matrix addition as a simple case of two-dimensional parallel computation. CUDA provides built-in support for organizing threads and blocks in one, two, or three dimensions, which naturally fits data structures like arrays and matrices. The example is shown Listing 4 and the full code can be found in `skeleton_codes/mat_add`. The key

elements are:

- The keyword `global` declares the function as a CUDA.
- Compute global thread indices `i` (row) and `j` (column) using CUDA built-in variables `blockIdx`, `blockDim`, and `threadIdx`.
- The condition `if (i < N && j < N)` prevents out-of-bounds memory accesses.
- Each thread computes one matrix element `C[j * N + i]` by adding corresponding elements from `A` and `B`.
- Matrix elements are stored in column-major order.

```
__global__ void mat_add(const float *A, const float *B, float *C, int N) {
  // Compute index with CUDA built-in variables blockIdx, blockDim and threadIdx
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int j = blockIdx.y * blockDim.y + threadIdx.y;
  // Avoid out-of-bound
  if (i < N && j < N) {
    C[j * N + i] = A[j * N + i] + B[j * N + i]; // column-major ordering
  }
}
```

Listing 4: CUDA kernel for addition of two matrices. Each thread computes one element of the result matrix by determining its global index from the grid and block dimensions.

The snippet Listing 5 shows how to configure and launch the `mat_add` kernel for matrix addition using a two-dimensional grid and block layout:

- Define a block of `16 x 16` threads using CUDA's `dim3` ([5, Sec. 5.4.2.2.[12]]) type.
- Compute the number of blocks in each grid dimension to cover all `N x N` matrix elements, rounding up to ensure full coverage.
- Each block of `threads.x * threads.y` threads executes on a SM.
- The grid consists of `blocks.x * blocks.y` blocks of threads, which execute concurrently on the device.
- The kernel `mat_add` is launched with the computed grid and block dimensions.

```
  dim3 threads(16, 16);
  dim3 blocks((N + threads.x - 1) / threads.x,
              (N + threads.y - 1) / threads.y);
  // "threads.x * threads.y" threads will execute mat_add as a block on a SM
  // "blocks.x * blocks.y" blocks of threads execute as a grid on the device
  mat_add<<<blocks, threads>>>(d_A, d_B, d_C, N);
```

Listing 5: CUDA kernel for addition of two matrices. Each thread computes one element of the result matrix by determining its global index from the grid and block dimensions.

CUDA code is again compiled with NVIDIA's `nvcc` compiler and run as follows:

```
user@eu-login-12:~$ module load stack gcc cuda # load a stack, a compiler and CUDA
user@eu-login-12:~$ nvcc -arch=sm_75 mat_add.cu -o mat_add # compile for CC 7.5
user@eu-login-12:~$ srun --gpus=rtx_2080:1 ./mat_add # Run command with a job
                                                     # requiring a specific GPU
srun: job 31822488 queued and waiting for resources
srun: job 31822488 has been allocated resources
Results match! GPU computation is correct :-).
```

[12] https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#built-in-variables

2.2 Kernels with synchronization

In previous examples, each thread operated independently on its own data element. However, many algorithms require threads to communicate or coordinate to produce a correct result. CUDA provides synchronization mechanisms for this purpose.

Two common constructs are:

- **Barrier synchronization**: The function `__syncthreads()` ([5, Sec. 5.4.4.[13]]) synchronizes all threads within a block. Threads are paused at this point until all threads in the block have reached the barrier. It is commonly used together with *shared memory* (see, e.g., [5, Sec. 2.2.3.2.[14]]) to enable thread cooperation and data exchange within a block.
- **Atomic operations**: Atomic functions ([5, Sec. 5.4.5.[15]]), such as `atomicAdd()` ([5, Sec. 5.4.5.1.1.[16]]) or `atomicMax()` ([5, Sec. 5.4.4.1.9.[17]]), ensure that read-modify-write operations on shared data are performed without race conditions, even across multiple threads.

Reduction operators[18] are prototypical examples needing synchronization and we will look at a simple application.

2.2.1 Maximum in vector kernel

As an example, we now consider how to compute the maximum value of an integer vector. A straightforward, but naive, solution assigns one thread per element. This solution is displayed in Listing 6. Each thread performs an atomic update to a shared global variable holding the maximum value:

- Each thread reads one element of `x[i]`.
- An atomic update with `atomicMax()` to `max_val` prevents race conditions.

This approach is simple but inefficient: all threads contend for the same memory location with the atomic operation.

```
10 __global__ void vec_max_naive(const int *x, int *max_val, int N) {
11   int i = blockIdx.x * blockDim.x + threadIdx.x;
12
13   if (i < N) {
14     atomicMax(max_val, x[i]);
15   }
16 }
```

Listing 6: CUDA kernel for maximum in vector: Naive version using an atomic for every element.

A simple optimization shown in Listing 7 reduces overhead by allowing each thread to process multiple elements using a so-called *grid-stride loop*:

- Compute the global thread index `i` using `blockIdx.x`, `blockDim.x`, and `threadIdx.x`.
- Compute the stride as the total number of threads in the grid: `stride = blockDim.x * gridDim.x`.
- Initialize a thread-local variable `max_local` with `INT_MIN` to track the maximum found by this thread.
- Use a grid-stride loop (`while (i < N)`) to process multiple elements assigned to this thread.
- For each element `x[idx]`, update `max_local` with the larger value.

[13] https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#synchronization-primitives
[14] https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#shared-memory
[15] https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#atomic-functions
[16] https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#atomicadd
[17] https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#atomicmax
[18] https://en.wikipedia.org/wiki/Reduction_operator

- After processing, perform a single `atomicMax()` to update the global maximum `max_val` with the thread's local result.

The number of launched threads is independent of the vector size N. By choosing fewer threads and letting each thread process multiple elements, the number of atomic operations is reduced further, while still covering the entire vector.

```
__global__ void vec_max_gridstride(const int *x, int *max_val, int N) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int stride = blockDim.x * gridDim.x;

  int max_local = INT_MIN;

  while (i < N) {
    max_local = max(max_local, x[i]); // using max in CUDA API
    i += stride;
  }

  atomicMax(max_val, max_local);
}
```

Listing 7: CUDA kernel for maximum in vector: Grid-stride loop version to reduce the number of atomics.

However, the grid-stride loop represents only a minor optimization and there are much more advanced techniques. We recommend part 5 on "Atomics, Reductions, and Warp Shuffle" of the CUDA training series [7] for further studies beyond this course.

2.3 Implement the simple kernels [30 Points]

Your task is to implement all the simple linear algebra kernels in `simple_kernels/linalg.cu`:

- `hpc_fill`,
- `hpc_axpy`,
- `hpc_add_scaled_diff`,
- `hpc_scale`,
- `hpc_lcomb`,
- `hpc_copy`.

The main provided in `benchmark.cu` runs all the kernels and reports the measured performance and bandwidth. Briefly comment and report the results for FP formats `float` and `double` (see the `real_t` typedef) in a roofline plot (i.e., put appropriately labeled points into a roofline model plot for each kernel).

2.4 Implement the scalar product kernel [20 Points]

Your task is to implement the dot product kernel in `dot_product/dot_product.cu`. Briefly comment and report the results for FP formats `float` and `double` (see the `real_t` typedef) in a roofline plot (i.e., put appropriately labeled points into a roofline model plot for each kernel).

Bonus [10 Points]: Optimize you dot product implementation using more advanced techniques.

3 Non-linear PDE mini-app with CUDA: The Fisher equation [35 Points]

We are now ready to port the familiar mini-app Fisher equation PDE solver to GPUs. You will find the skeleton code in `mini_app`, where much of the code is already implemented. Complete the following tasks:

- Implement the linear algebra kernels in `linalg.cu`.
  Hint: You can simply reuse your kernels from Sections 2.3 and 2.4.

- Implement the stencil operator for the finite difference discretization in `operators.cu`.
  Hint: Study the matrix add example in Section 2.1.2.

Make sure that your code compiles and produces correct results for double precision (`real_t` set to `double` in `data.h`). In your report, simply include a plot of the population density at initial and final times.

Bonus [10 Points]: Make the solver min-app also work for single precision (`real_t` set to `float` in `data.h`) and document your solution.

A Additional notes and submission details

Submit your **report** and **all source code files**, including your build files (e.g., `Makefile`(s)) and other scripts (e.g., batch job scripts, plotting scripts), as a compressed tar archive. Your LaTeX report must be based on the provided template from the course webpage and should summarize your results and observations for all sections. Your submission should be uploaded to SAM-UP and must include

- Your report, named as `project_number_lastname_firstname.pdf`. You may discuss all questions with anyone you like. However, (i) you must list all collaborators in your submission, and (ii) you must prepare and write up the submission independently.
- A compressed tar[19] archive, named `project_number_lastname_firstname.tar.gz`, containing:
  - All source code files.
  - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
  - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
- The total size of your submission (report PDF and archive) must not exceed 25 MB.
- File names must contain only `ASCII` characters (avoid spaces, special symbols, and non-English letters).

Please strictly follow these instructions and naming conventions. Failure to do so results in additional work for the teaching assistants.

References

[1] NVIDIA Corporation. CUDA legacy GPUs compute capability table. https://developer.nvidia.com/cuda-legacy-gpus, 2025. Accessed: 2025-05-12.

[2] NVIDIA Corporation. CUDA C++ best practices guide. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/, 2026. Version 13.2; Accessed: 2026-05-08.

[3] NVIDIA Corporation. CUDA C++ programming guide. https://docs.nvidia.com/cuda/archive/12.9.0/cuda-c-programming-guide/, 2026. Version 12.9.0; Accessed: 2026-05-08.

[4] NVIDIA Corporation. CUDA devicequery – device query. https://github.com/NVIDIA/cuda-samples/tree/master/Samples/1_Utilities/deviceQuery, 2026. Accessed: 2026-05-07.

[5] NVIDIA Corporation. CUDA programming guide. https://docs.nvidia.com/cuda/cuda-programming-guide/, 2026. Version 13.2; Accessed: 2026-05-08.

[6] NVIDIA Corporation. *CUDA Toolkit Documentation: Runtime API*, 2026. Version 13.2.1; Accessed: 2026-05-08. URL: https://docs.nvidia.com/cuda/cuda-runtime-api/index.html.

[7] NVIDIA Corporation. Cuda training series at olcf. https://www.olcf.ornl.gov/cuda-training-series/, 2026. Accessed: 2026-05-07.

[8] TOP500.org. Frequently asked questions: What is the theoretical peak performance of a system? https://www.top500.org/resources/frequently-asked-questions/#_What_is_the_theoretical%20peak%20perfor, 2026. Accessed: 2026-05-07.

[19] https://en.wikipedia.org/wiki/Tar_(computing)